

Module 08 — Hybrid Recommender

models/hybrid_recommender.py • 411 lines

In one sentence

Combines four signals into one ranked list. This is the best-performing model.

1. What this module does

Each individual model has a blind spot. Collaborative filtering is strong for established customers but silent on new products. Content-based is the opposite - it works on day one but never discovers that two unrelated-looking products appeal to the same people. The deep model captures patterns neither can express, but needs history on both sides.

The hybrid runs all four, rescales their scores to a common 0-1 range, and combines them with weights. The result beats every individual model.

It is written as a reusable **service class** rather than a script, because both the API and the dashboard load exactly this object. Neither contains any scoring code of its own, so they can never disagree.

2. Concepts covered

Every idea this module uses, in plain terms.

Concept	In simple terms	Where it is used here
Hybrid recommendation	Combine several models into one	The whole module
Score fusion	Blend scores with weights	<code>fuse()</code>
Weighted linear combination	$w1*a + w2*b + w3*c + w4*d$	The fusion formula
Score normalisation	Put all signals on the same scale first	<code>min_max_normalize</code> on each signal
Weight renormalisation	Rescale if a signal is missing	Handles unknown customers
Cold-start fallback	A different strategy when there is no history	<code>cold_start_recommendations()</code>
Strategy labelling	Say which path produced the answer	The strategy column
Service class pattern	Load once, reuse many times	<code>HybridRecommender.load()</code>
Caching / precomputation	Do slow work once at startup	<code>user_seen</code> dictionary
Graceful degradation	Keep working if a part is missing	Runs without NCF, but says so loudly

3. How it works, step by step

- 1. Load everything once** - matrices, similarity grids, the trained NCF, and the index maps.
- 2. Precompute each customer's history** into a dictionary, so no request has to scan the 138,345-row log.
- 3. On a request, check for cold start first.** Fewer than 3 interactions means the fallback path, not the models.
- 4. Otherwise compute four scores** - item-based CF, SVD, content, and NCF - each rescaled to 0-1.
- 5. Blend them** using the weights, renormalising if any signal produced nothing.

6. **Re-rank the top 200** for popularity balance.
7. **Attach the catalogue details and a strategy label**, and return.

4. Key functions

Method	What it does
HybridRecommender.load()	Loads all artifacts, warns loudly if NCF is missing
is_cold_start_user()	Under 3 interactions?
item_cf_scores()	Item-based CF signal - the strongest
collaborative_scores()	SVD latent-factor signal
content_scores()	TF-IDF taste-profile signal
ncf_scores()	Deep learning signal
fuse()	Weighted blend plus re-ranking
cold_start_recommendations()	Two-tier fallback
recommend()	The main entry point
similar_items()	Content neighbours for a product

5. Inputs, outputs and how to run

	Detail
Inputs	All model artifacts from data/processed
Outputs	A DataFrame of recommendations with per-signal scores
Result	NDCG@10 = 0.1095 - best of every model tested
Load time	A few seconds (several hundred MB of matrices)
Run with	python models/hybrid_recommender.py

How the weights were chosen - by measurement, not intuition

Configuration tried	NDCG@10
item-based CF on its own	0.1245
svd + content + ncf (the first version)	0.0947
item_cf + content + ncf	0.1282
item_cf .50 / content .10 / ncf .40	0.1276
item_cf .45 / svd .10 / content .10 / ncf .35 - chosen	0.1293
item_cf .70 / ncf .30 (no content)	0.1181

6. Key code

models/hybrid_recommender.py — HYBRID_WEIGHTS()

```
30 | HYBRID_WEIGHTS = {
31 |     "item_cf": 0.45,
32 |     "collaborative": 0.10,
33 |     "content": 0.10,
34 |     "ncf": 0.35,
35 | }
```

models/hybrid_recommender.py — HybridRecommender.fuse()

```
229 | def fuse(self, item_cf, collaborative, content, ncf) -> pd.DataFrame:
230 |     """Weighted linear fusion, then popularity-balanced re-ranking."""
231 |     available = {
232 |         "item_cf": item_cf,
233 |         "collaborative": collaborative,
234 |         "content": content,
235 |         "ncf": ncf,
236 |     }
237 |     active = {name: s for name, s in available.items() if not s.empty}
238 |
239 |     if not active:
240 |         return pd.DataFrame()
241 |
242 |     # Renormalise over active signals, otherwise a user the NCF has never
243 |     # seen gets scores scaled by the surviving weight mass and they stop
244 |     # being comparable across users.
245 |     weight_mass = sum(self.weights[name] for name in active)
246 |     effective_weights = {name: self.weights[name] / weight_mass for name in active}
247 |
248 |     all_item_ids = sorted(set().union(*(set(s.index) for s in active.values())))
249 |     fused = pd.DataFrame(index=all_item_ids)
250 |
251 |     for name, series in available.items():
252 |         fused[name + "_score"] = series.reindex(fused.index).fillna(0.0)
253 |
254 |     fused["hybrid_raw_score"] = sum(
255 |         effective_weights[name] * fused[name + "_score"] for name in active
256 |     )
257 |
258 |     reranked = rerank_with_popularity_balance(
259 |         fused["hybrid_raw_score"],
260 |         self.popularity_lookup,
261 |         candidate_pool=CANDIDATE_POOL_SIZE,
262 |     )
263 |
264 |     fused = fused.loc[reranked.index].join(
265 |         reranked[["adjusted_score", "interaction_count", "is_long_tail",
266 |                 "popularity_penalty", "long_tail_boost"]]
267 |     )
268 |
269 |     fused["active_signals"] = ", ".join(sorted(active))
270 |     return fused.sort_values("adjusted_score", ascending=False)
```

Note the renormalisation step when a signal is missing.

7. Things to remember

The hybrid was originally worse than its own input

The first version blended SVD, content and NCF and scored 0.0947 - **worse than plain item-based CF on its own at 0.1245**. The cause was simple once measured: the collaborative slot was filled by SVD, the weakest collaborative signal, while item-based CF, the strongest model in the whole system, was not in the fusion at all. Swapping it in took the score to 0.1293.

Why rescale each signal before blending

The four signals produce numbers on completely different scales. Without rescaling to 0-1 first, whichever signal happens to produce the biggest raw numbers would dominate, regardless of the weights you set.

Why every response carries a strategy label

A caller must always be able to tell a genuinely personalised recommendation from a cold-start fallback. Presenting the two identically is how a dashboard ends up claiming a brand-new customer has a learned taste profile.

8. Likely questions

Q. Why does combining models help?

A. Because their errors are different. Item-based CF is blind to new products; content-based is blind to non-obvious taste patterns; NCF needs history on both sides. Blending means one signal can cover for another's blind spot. Measured, the hybrid at 0.1095 beats its own strongest component at 0.1044.

Q. How did you pick the weights?

A. I swept several combinations over a 250-customer cohort and measured NDCG@10 for each. The chosen setting scored highest at 0.1293. I did not pick them by intuition - the first intuitive guess scored worse than a single model.

Q. Why keep content at only 10% when it scores 0.0116 alone?

A. Because it has 75.4% catalogue coverage against everything else's 20%. It buys reach into the long tail. Removing it drops the overall NDCG, so it is earning its place even though it is weak on its own.

Q. What happens if the NCF model file is missing?

A. The service still starts, prints a loud warning naming the exact file and the command to fix it, and renormalises the remaining three weights so scores stay comparable. The previous version of this project silently served with the deep signal switched off and reported itself healthy.